Designing Agent Outputs for Fast Comprehension

Research synthesis and implementation plan for the ECC Focus Queue output system · 2026-10-05 · Prepared for the system-prompt and Focus Queue integration work that follows

> **BOTTOM LINE UP FRONT**
>
> Your five-part structure (BLUF → what/why/how per step → big-picture fit → QA instructions → next steps) is directionally correct and well supported by the evidence. The research adds four things it is missing: **(1)** a stage badge so the orchestrator always knows where a task sits in the development pipeline, **(2)** mandatory evidence per claim (command + result, never prose assertions) plus an explicit "not verified" list — the single strongest counter to agent overconfidence, **(3)** a ranked rule set for sentence- and layout-level writing, where the highest-yield rule is deleting everything that carries no information, and **(4)** enforcement mechanisms, because prompting alone does not fix sycophancy — structure does.

What A research-backed design for how AI coding agents should format their output, plus a phased plan to implement it as a system-prompt insert integrated with Focus Queue.

Why You orchestrate multiple agents with a small attention budget; current outputs are long, self-favorable, and hard to verify. Comprehension research and AI-oversight research converge on the same fixes.

How Three parallel research streams (ADHD/plain-language guidelines, comprehension-and-retention science, AI-agent oversight practice) were synthesized against your existing brainstorm; every recommendation below is tagged with its evidence strength and source.

---

1. The problem, stated precisely

An orchestrator running several agents must do three things fast, repeatedly: **re-ground** (what is this task, where is it), **evaluate** (is the work correct and complete), and **decide** (approve, redirect, kill, or move on). Agent output today works against all three: it is long, mixes conclusion with process, asserts success in prose, and buries

the verification surface. Two findings from the research frame why this is urgent rather than cosmetic:

- **Automation complacency peaks in exactly your configuration.** The complacency literature (Goddard et al.; Parasuraman & Manzey) identifies three preconditions — a highly-but-not-perfectly reliable system, high workload, and multitasking. Managing N parallel agents is the textbook worst case. Design, not discipline, is the fix.
- **Sycophancy is trained in, not prompted out.** Preference training rewards agreement and confidence (Sharma et al., five frontier models). Asking the model nicely to be honest has limited effect; requiring evidence fields and independent grading has structural effect.

ADHD-relevant mechanism: adult ADHD reading-comprehension deficits trace to **sustained attention and working memory**, not vocabulary or decoding (Segal 2023; Kofler et al. 2019). So the goal is not simpler words — it is lower memory load and attention cost per unit of text, with state externalized onto the page.

---

2. What the research says: presentation rules

Evidence grades: **Strong** = meta-analyses or large replicated literatures. **Moderate** = replicated studies with boundary conditions. **Practice** = professional convention, thin direct evidence.

2.1 Highest-yield rules, ranked by evidence weight

#	RULE	EVIDENCE	KEY SOURCE
1	**Cut everything that carries no information.** No filler, no restating the prompt, no narrating what a table already shows, no "great question." This is the single largest measured effect on learning (coherence principle, g ≈ 1.0). Each fact appears once, in its best form.	**Strong**	Mayer / Cromley & Chen 2025; Chandler & Sweller
2	**Conclusion first, then labeled sections.** BLUF, then fixed labeled chunks. Structured (labeled) summaries measurably beat unstructured prose; signaling via headings improves recall, and *disproportionately helps taxed readers*.	**Strong**	Hartley; Lorch & Lorch; Naumann et al. 2007
3	**Concrete specifics, never abstractions.** "writes to `Config/ecc.config.json`" beats "updates the configuration." Concrete referents are recalled reliably	**Strong**	Concreteness effect (Paivio);

	better; exact paths and IDs also act as retrieval cues later.		Dunlosky et al. 2013
4	**Information-dense lead words in headings and bullets.** Readers follow "information scent" and scan the left edge; headings like "Overview" or "Details" are dead zones. Headings must name outcomes, files, decisions.	**Strong**	NN/g F-pattern & information foraging
5	**Direct, conversational tone.** "I changed X" not "the modification of X was performed." Active voice, first/second person (personalization, g ≈ 0.7).	**Strong**	Mayer; plainlanguage.gov; CDC Index
6	**One structural table or diagram when ≥3 parts interact.** Dual coding: a table carrying real structure beats a paragraph (g ≈ 0.68). But never narrate what the visual already shows (redundancy effect *hurts* learning).	**Strong**	Mayer; Chandler & Sweller 1991
7	**Define components before mechanism.** A one-line "components involved" note before a dense change description (pre-training principle, median effect ≈ 0.75).	**Moderate**	Mayer pre-training; CORE review
8	**Keep annotations adjacent to their referent.** File path next to the claim about the file; decision next to the diff. Forcing the reader to integrate separated sources costs comprehension (split-attention effect).	**Strong**	Sweller CLT
9	**End with a short self-verification checklist.** Retrieval beats rereading (61% vs 40% recall after one week); asking the reader to check or predict engages generative processing.	**Strong**	Roediger & Karpicke 2006
10	**One worked example inside the rules.** A complete annotated example of a good report teaches the format better than the rules alone (worked-example effect).	**Strong**	Sweller / Cooper

2.2 Sentence- and layout-level rules (style-guide conventions, institutionally backed)

- **One idea per sentence.** More than one conjunction or comma → split. Hard limit ≈25 words, average target 15–20. (plainlanguage.gov; GOV.UK/ONS; ASD-STE100)

- **One idea per paragraph; ≤4 sentences.** A one-sentence paragraph is valid. First sentence of each paragraph carries the point — a reader scanning only first sentences should still get the gist. (W3C COGA; NN/g inverted pyramid)
- **Lists over prose; number steps, bullet options.** Scannable formatting alone gave +47% measured usability; concision +58%; objective tone +27% — and the effects compound (+124% combined). (NN/g 1997)
- **Descriptive headings, not clever or generic ones.** Avoid question headings (slower to scan) and print-era labels ("Introduction," "Background").
- **No double negatives, no metaphors or idioms, simple tense.** Define jargon once, then use one term consistently — do not paraphrase-repeat. (W3C COGA)
- **Bold key words sparingly.** Overuse reads as shouting and destroys the signal. (W3C COGA)
- **Externalize state; never make the reader hold context.** Restate ticket, stage, and goal in the output itself. Assume the reader just reopened this session cold. (W3C COGA "restore context"; St Andrews ADHD guidance)
- **Progressive disclosure.** Fixed short layer always visible (BLUF + what/why/how); detail available but never interleaved with the summary. (Nielsen 2006; St Andrews)

> **HONEST EVIDENCE NOTE**
>
> No controlled studies test specific formatting rules on adults with ADHD. The ADHD-specific guidance is practitioner consensus extrapolated from (a) general usability research and (b) cognitive-accessibility work, justified by the established mechanism (working-memory and sustained-attention load). The strategy is therefore: apply the load-reduction rules that help everyone, weighted harder. The numeric thresholds (25-word sentences, 15-word averages) are style-guide conventions — the most defensible numbers available, not ADHD-validated constants.

---

3. What the research says: verification and staying on track

3.1 The completion report must show evidence, not assertions

- **Evidence per claim.** Anthropic's guidance: agents stop when work "looks done," so every completion claim needs a verifiable artifact — the command run and what it returned, test counts, a screenshot. "Tests: 47 pass, 0 fail, 3 skipped" calibrates trust; "I'm confident this works" does not. Reviewing evidence is faster than re-running verification.

- **A mandatory “not verified” list.** The strongest single anti-overconfidence device: the agent must explicitly name what it did *not* check. If verification belongs to a later stage, it says so and names the stage — this resolves your open question: **state the gap, don't fill it in silently.**
- **Calibrated confidence only.** Miscalibrated confidence is worse than none (arXiv 2402.07632). Confidence must be tied to evidence (“tested” / “untested area”), never a feeling. Uniform confidence trains blanket acceptance.
- **Never let the worker grade itself.** A fresh-context reviewer subagent that tries to refute the result removes self-grading bias. Symmetric failure mode: reviewers prompted to find gaps always find some — severity-filter findings to correctness and stated requirements only.

3.2 Stage as state, not prose

- **Artifacts define stages.** Idea → spec → plan → diff → test evidence → review. Each stage produces a durable artifact; stage is derived from which artifacts exist, never from the agent's self-report. (Anthropic; Google Jules exposes explicit session-state enums.)
- **Definition of done is written before implementation.** The spec ends with an end-to-end verification step, so completion claims are checkable against a pre-registered criterion.
- **The report maps to your pipeline:** idea/concept → planning → implementation → testing → quality check/refinement → deployment → deployed-verification. Every output opens by naming its stage and the artifact that proves it.

3.3 Review-surface design

- **CL-description standard.** Google's code-review practice is the canonical answer to “agent outputs are too long”: a standalone first line stating *what*, a body stating *why* plus known shortcomings, evidence attached. “Fix bug” and “Phase 1” are explicitly anti-patterns.
- **Small review units.** Defect detection peaks at 200–400 lines per review and collapses beyond ~400; effectiveness drops after 60–90 minutes. Agent tasks should be scoped so each produces a reviewable diff — never a multi-thousand-line dump.
- **Decision rules, not vibes.** Named failure patterns give the orchestrator crisp actions: “if you can't verify it, don't ship it”; more than 2 corrections → restart with a better prompt rather than keep correcting.
- **Design against complacency.** Vary scrutiny by confidence; use targeted confirm questions (“this assumes X applies — correct?”) instead of blanket

approval; track acceptance rates — if high- and low-confidence outputs are accepted at the same rate, complacency has set in.

---

4. Integration: the output contract

Your five-part structure survives contact with the research almost intact. The synthesis below keeps your skeleton, adds the stage badge and the evidence/not-verified machinery, and converts your open questions into defaults. This is the format the system-prompt insert will specify.

0 · Stage badge **(always, one line)**
`[Stage: implementation → testing] [Task: ECC-123 short-name] [Artifact: Tests/FooTests.swift, 12 pass]`
Names where the task sits in the pipeline and the artifact that proves it. Stage is derived from artifacts, not claimed.

1 · BLUF **(always, 1–2 sentences)**
The result and the one thing the user must know or do. Re-grounds a cold reader in under ten seconds.

2 · What / Why / How **(always, one short paragraph)**
What changed — concrete: exact paths, IDs, values. **Why** — the line connecting change to request. **How we know** — the evidence: command run + result. Your existing FocusQ structure; unchanged, because it is the structured-abstract pattern with the strongest direct support.

3 · Decision log **(per major step or design choice; skip trivia)**
For each: **What** was decided · **Why** (reasoning + rejected alternatives when they exist) · **How verified** (evidence, or explicit "not verified — verification planned in testing stage"). Justification is mandatory *precisely because* forcing the agent to produce it catches errors before they ship — your instinct is confirmed by the evidence-guided-prompting finding that requiring evidence before verdict measurably reduces wrong, self-favorable judgments.

4 · Big-picture fit **(conditional — only when load-bearing)**
Why this integrates with the whole: branding checklist + Playwright sanity checks (your UI example), or determinism + cost-rate math against the PRD (your backend example). **Default: omit.** Include only when the change touches architecture, multiple subsystems, or deviates from plan. Rule 1 (cut what carries no information) outranks completeness; per-step justifications usually already cover it — your own doubt here was correct.

5 · QA instructions **(scaled to task, but near-always present)**
The fastest path to production-like verification: exact commands, what to observe, expected result. Scales from "tell me if you like how it looks" to a full sandbox

walkthrough. Written so a cold reader can execute without reconstructing context. Ends with a short check-yourself list when the change is behavioral.

6 · Next steps **(always unless task is verifiably done)**
What remains before "done," concerns, discoveries that open new routes, and the decision the user faces next, if any.

4.1 Standing language rules (apply to every section)

1. Sentences: one idea, ≤25 words, active voice, direct ("I changed X").
2. Concrete referents: paths with line numbers (`Sources/FocusQueueCore/Queue.swift:42`), ticket IDs, exact values.
3. Lead words carry information: every heading, bullet, and paragraph starts with the operative content.
4. Lists over prose. One structural table/diagram when ≥3 parts interact; never duplicate it in prose.
5. Each fact stated once. No filler, no restating the request, no self-praise.
6. Define components before mechanism when the reader may not hold them in memory.
7. Claims about correctness always attach evidence or an explicit "not verified" marker.

---

5. Enforcement: making justification actually happen

Your open question — how to enforce justification rather than hope for it — has a two-part answer from the research: **required fields with explicit nulls**, and **structural independence between worker and grader**.

1. **Format as fields, not vibes.** The template is a contract: every section is present or explicitly marked `N/A — reason`. An absent justification is a visible defect, not a stylistic choice. This converts honesty from a behavior the model must choose into a hole it must explain.
2. **Evidence-before-verdict ordering.** The "How verified" field physically precedes any completion claim. Evidence-guided ordering measurably reduces self-favorable judgments; it also operationalizes your observation that forcing justification prevents errors — the agent must confront the evidence before writing the claim.
3. **The "not verified" list is mandatory and specific.** Not "some edge cases untested" but "error path for empty config untested; covered in testing stage." Gaps name the stage that will close them.

4. **Fresh-context verifier for high-stakes completions.** A lightweight second agent reviews the report against the diff and tries to refute it; findings are severity-filtered (correctness and stated requirements only) so the human is not drowned.
5. **Pre-registered definition of done.** The planning stage writes the end-to-end verification step; the completion report is checked against it. This makes “done” a comparison, not an adjective.
6. **Small-diffs scoping.** Tasks are sized to produce 200–400-line review units so the human's review is physically feasible.
7. **One worked example in the prompt.** The system-prompt insert contains a complete annotated example report. Models imitate format from examples far more reliably than from rules alone (worked-example effect).
8. **Complacency metrics (Focus Queue layer).** Track acceptance rate by confidence level and time-to-accept. Flattening acceptance rates trigger increased friction selectively, per the trust-adaptive-intervention pattern.

6. Implementation plan

PHASE	DELIVERABLE	VERIFICATION	DEPENDS ON
1 · Prompt insert	The output contract (§4) + language rules (§4.1) + one worked example, as a system-prompt module for Codex/ Claude entry points (`Scripts/focus_queue_*_entry.py`).	Run 3–5 representative tasks (bugfix, feature, research) through both old and new prompts; score outputs against the contract field-by-field.	—
2 · Stage state	Stage badge wired to Focus Queue task state: stage derived from artifacts (spec file, plan, diff, test results) registered per ticket, matching your task-session association flow.	Stage displayed in Focus Queue matches artifact reality for a live task across at least three stage transitions.	Phase 1
3 · Detail extraction	Connect to the planned proactive extraction system: full decision logs, diffs, and evidence are extracted to	Given a completed session, every claim in the	Phase 1, extraction system

	task storage; chat output carries the summary layer plus pointers. This is progressive disclosure implemented at the data layer — no detail lost, summary never polluted.	summary resolves to a stored artifact; the summary itself stays within length limits.	
4 · Verifier pass	Lightweight fresh-context verifier for completion reports on high-stakes tasks, with severity filtering.	Verifier catches a planted error in a sample report; false-positive rate low enough that findings fit in 3 bullets.	Phase 1
5 · Metrics	Acceptance-rate-by-confidence and time-to-accept logging in Focus Queue; complacency-flattening alert.	Dashboard shows the two metrics for a week of real usage.	Phase 2

Phases 1 and 2 are the core and stand alone. Phases 3–5 compound the value but each is independently skippable.

---

7. Discussion: things to experiment with

- **A/B the contract against current outputs.** Take five completed sessions, re-render their final reports in the new format, and time yourself answering: what changed, is it verified, what do I do next. This is cheap and tells you if the format pays before any enforcement machinery is built.
- **“Where am I likely wrong?”** A required self-critique line in the decision log for high-stakes choices. Promising from the sycophancy-mitigation literature, but risks boilerplate; test whether answers stay specific.
- **Targeted confirm questions.** Instead of “please review,” the agent asks one specific question about its weakest assumption (“this assumes the config schema didn't change — correct?”). Intentional friction that forces engagement at exactly the uncertain point.
- **Diagram rule calibration.** The “≥3 interacting parts → one structural visual” threshold is a guess. Watch where ASCII diagrams help vs. where they become decoration (which violates rule 1).
- **Confidence vocabulary.** Two levels (tested / not tested) are calibratable; a five-point scale is not. Resist richer scales until metrics show the coarse one is limiting decisions.

- **Length budget.** A hard budget (e.g., BLUF + What/Why/How ≤ 100 words) forces the compression that rule 1 demands. Worth testing whether agents honor it or route around it with dense fragments.
- **Session-reopen regrounding.** Since BLUF's job is re-grounding a cold reader, test the first screen of every report against "10-second rule": can you state task, stage, result, and next action after ten seconds?

---

8. Sources

1. Nielsen Norman Group — Inverted Pyramid; Chunking; F-Pattern; Information Foraging; Concise/Scannable/Objective (1997). nngroup.com/articles/
2. W3C Cognitive Accessibility (COGA) Techniques. w3c.github.io/coga/techniques/
3. plainlanguage.gov guidelines; CDC Clear Communication Index; National Archives plain-writing principles.
4. GOV.UK / ONS content-design manuals; ASD-STE100 Simplified Technical English.
5. University of St Andrews — Designing for users with ADHD (2025). digitalcommunications.wp.st-andrews.ac.uk
6. Segal (2023) sustained attention & reading comprehension; Kofler et al. (2019) working memory; SAGE scoping review (2021).
7. Mayer multimedia learning; Cromley & Chen (2025) meta-analysis, ScienceDirect S1747938X25000673.
8. Sweller Cognitive Load Theory; worked-example, split-attention, redundancy effects (Chandler & Sweller 1991/96).
9. Dunlosky et al. (2013) effective learning techniques; Roediger & Karpicke (2006) testing effect; concreteness effect (Paivio).
10. Lorch & Lorch (1993/96) topic-structure signaling; Naumann et al. (2007) signaling compensates for reading skill.
11. Hartley (1997–2003) structured abstracts; Wager review, PMC8958823.
12. Anthropic — Claude Code Best Practices. anthropic.com/engineering/claude-code-best-practices
13. Google eng-practices — CL descriptions & small CLs. google.github.io/eng-practices/
14. Sharma et al. — sycophancy in RLHF models, arXiv 2310.13548; sycophancy & trust, arXiv 2412.02802; evidence-guided prompting, arXiv 2607.10411.
15. Automation bias systematic review, PMC3240751; Parasuraman & Manzey complacency literature.
16. Trust calibration: Lee & See lineage; arXiv 2402.07632 (miscalibrated confidence); UXmatters "Designing for Doubt"; LoopRails HITL framework.
17. Google Jules docs (session states, plan-approval gate); Osmani — conductors to orchestrators; Codex best-practice guides.
18. SmartBear/Cisco code-review study lineage (200–400 LOC review units).